



CLAUSIFY

AI Legal Agreement Scanner & Consumer Grievance Assistant

Comprehensive System Architecture, 5 Development Phases & Technical Reference Report | Generated: October 2026

Executive Overview: Clausify is an end-to-end legal intelligence platform built to protect consumers against unfair contract terms, unilateral contract alterations, hidden recurring charges, and invasive data harvesting. It pairs dense local vector embeddings with high-speed LLM reasoning to evaluate agreements, ground live conversational dialogue, and compile formal dispute notices in accordance with modern consumer protection statutes.

1. SYSTEM ACCESS POINTS & NETWORK TOPOLOGY

The platform runs three interconnected background services ensuring seamless local access across web interfaces and developer endpoints:

Service / Interface	Address / URL	Default Port	Functional Purpose
Unified Gateway Proxy	http://localhost:8080	8080	Unified Entry Point: Dynamically routes UI requests to port 5173 and API/Swagger requests to port 8000.
Interactive Web UI	http://localhost:5173	5173	Vite + React 19 Frontend application featuring dynamic risk visualizers and live chat.
Core FastAPI Server	http://localhost:8000	8000	High-performance ASGI backend serving ingestion, AI inference, and notice generation.
Swagger UI (Docs)	http://localhost:8000/docs	8000 / 8080	OpenAPI 3.1 interactive testing console for all endpoints.
ReDoc Documentation	http://localhost:8000/redoc	8000 / 8080	Formal specification documentation view.
Operational Health Check	http://localhost:8000/health	8000 / 8080	JSON health heartbeat returning backend status, version, and active AI model names.

2. COMPLETE BREAKDOWN OF THE 5 DEVELOPMENT PHASES

Phase 1: Architecture, Environment & Core Foundation

Phase 1 established the infrastructural bedrock, external service integrations, and relational/vector storage schemas:

- Framework & Application Scaffolding:** Configured FastAPI with strict CORS middleware (`backend/app/main.py`), environment parsing via `python-dotenv` and `Pydantic` (`backend/app/core/config.py`).

- **Supabase PostgreSQL + pgvector Database:** Designed 3 core relational tables equipped with 384-dimensional vector columns:
 - `documents` : Tracks ingested PDFs, file names, timestamps, and overall aggregate risk scores.
 - `document_chunks` : Stores individual ~500-token text chunks, risk level ratings (`HI` , `MEDIUM` , `LOW` , `NONE`), plain-language flag reasons, and 384-d vector embeddings.
 - `faqs` : Pre-seeded curated legal questions and answers with 384-d vector embeddings for zero-latency retrieval.
- **Local In-Memory Vector Embedding Engine:** Integrated HuggingFace `sentence-transformers/all-MiniLM-L6-v2` in `backend/app/services/embedding_service.py` , producing normalized 384-dimensional dense vectors locally without cloud API costs or latency.
- **Groq AI Cloud SDK:** Initialized the high-speed inference client (`backend/app/core/groq_client.py`) supporting `openai/gpt-oss-120b` , `openai/gpt-oss-20b` , and `qwen/qwen3.8-27b` .

| Phase 2: Document Ingestion, PDF Processing & Admin Knowledge Base

Phase 2 implemented the end-to-end document intake and chunking pipeline alongside knowledge-base administrative capabilities:

- **PyPDF Extraction & Sanitization:** In `backend/app/api/routes_scan.py` , `extract_text_from_pdf` parses binary PDF data, decrypts blank password protections, and strips malformed characters.
- **Sliding-Window Token Chunking:** Implemented `split_into_chunks` utilizing a standard NLP token-to-word ratio (1 token is approx 0.75 words) to create ~500-token chunks with 50-token overlaps (~375 words per window, 38 words overlap) to ensure cross-boundary legal context retention.
- **Heuristic Keyword Pre-screening:** Evaluates clauses against predatory keywords ("*arbitrat*", "*class action*", "*unilateral modify*", "*personal data*", "*tracking*", "*auto-renew*", "*non-refundable*", "*indemnif*") to prioritize deep LLM analysis.
- **Admin FAQ CRUD API:** In `backend/app/api/routes_admin.py` , implemented `POST /api/v1/admin/faq` (vectorizes questions and stores them in Supabase) and `GET /api/v1/admin/faqs` for knowledge base updates.

Phase 3: Deep AI Clause Risk Analysis & Scoring Formulation

Phase 3 engineered the specialized LLM evaluation prompts, structured JSON schema outputs, and composite risk scoring formulation:

The 4 Primary Predatory Clause Categories Monitored:

1. **Forced Arbitration & Class Action Waivers:** Clauses stripping consumers of constitutional rights to open court jury trials or collective dispute aggregation.

2. **Unilateral Modifications:** Clauses permitting providers to alter prices, terms, or service scopes at sole discretion without advance notice.

3. **Third-Party Data Selling / Harvesting:** Broad permissions to monetize, trade, or transfer personal or behavioral data without explicit consumer consent.

4. **Auto-Renewal & Non-Refundable Payment Locks:** Involuntary subscription renewals paired with total zero-refund stipulations.

Aggregate Document Risk Formulation:

To balance overall breadth with the presence of critical terms, Clausify uses a weighted hybrid formula in `compute_overall_risk_score` :

$$\text{Risk Score} = (0.75 \times \text{Peak Severity Score}) + (0.25 \times \text{Average of Flagged Scores})$$

This prevents agreements with a single catastrophic clause (e.g. 95/100 arbitration waiver) from being hidden by benign introductory boilerplate, while adjusting higher if multiple violations exist.

Phase 4: Multi-Step Interactive RAG Chat & Legal Dispute Notice Engine

Phase 4 engineered the interactive conversational intelligence pipeline and document compilation engine:

Workflow Step	Technical Mechanism	Operational Value
Step A: FAQ Pre-Check	Cosine similarity check (<code>threshold >= 0.82</code>) against Supabase knowledge base vectors.	Returns verified legal guidance instantly without model latency or hallucinations.
Step B: Document RAG Grounding	Vector similarity search across active document chunks via <code>match_document_chunks</code> .	Forces the assistant to cite exact clauses; strictly refuses to fabricate missing contract terms.
Step C: Slot Entity Extraction	Analyzes intent and extracts 5 structured consumer grievance fields: <code>company_name</code> <code>transaction_id</code> <code>incident_date</code> <code>disputed_amount</code> <code>issue_summary</code>	Transforms unstructured complaints into actionable dispute parameters. When all 5 slots are populated, <code>notice_ready = True</code> is triggered.

Workflow Step	Technical Mechanism	Operational Value
Step D: Notice Compilation	Jinja2 template (<code>legal_notice.html</code>) compiled into binary PDF stream via <code>xhtml2pdf</code> .	Streams formal dispute notice citing the Consumer Protection Act 2019, EFTA, or UCC Section 2-302 directly to the user.

| Phase 5: Modern Interactive Web Application (React 19 + Tailwind CSS)

Phase 5 created a responsive, rich user experience (`frontend/src/App.jsx`) built around two modes:

- **Mode 1: Agreement Scanner:**
 - `DocUploader.jsx` : Drag-and-drop PDF uploader with live upload progress tracking and 3 pre-built sample contracts.
 - `RiskGauge.jsx` : Animated SVG radial gauge visually representing the 0–100 risk score with dynamic color grading.
 - `RedFlagList.jsx` : Categorized clause cards displaying plain-language summaries, violation reasons, and "Ask Assistant" shortcuts.
- **Mode 2: Grievance Notice Assistant:**
 - `ChatWindow.jsx` : Interactive conversation stream with verified FAQ chips, citation indicators, and quick prompts.
 - `GrievanceTracker.jsx` : Real-time visual slot progress tracker showing which details are confirmed, with editable inputs and a 1-click **Generate & Download Legal Notice (PDF)** action.
 - `LoginPage.jsx` : Multi-role authentication (Individual Consumer, Legal Advocate, Enterprise Compliance Officer).

3. TEST AGREEMENTS & VALIDATION ARTIFACTS

The workspace includes a generator script (`generate_sample_pdfs.py`) that outputs 3 agreements for demonstration and evaluation:

Sample Agreement	File Name	Expected Score	Key Clauses Included
StreamPlay Subscription	<code>streamplay_high_risk_agre</code>	85-95 CRITICAL	Forced binding arbitration, class action waiver, unilateral price modifications, and unrestricted data monetization.
CloudVault Terms	<code>cloudvault_moderate_risk</code>	45-55 MODERATE	Auto-renewal with notice, standard liability caps, and notice-based unilateral modifications.
FairDocs Agreement	<code>fairdocs_consumer_agreeme</code>	0-15 SAFE	Pro-rata refund policy, transparent dispute resolution in court, zero data selling, and explicit user consent for changes.

4. OPERATIONAL RUNBOOK & MAINTENANCE COMMANDS

Starting Services Manually (PowerShell):

1. Core Backend (FastAPI on Port 8000):
`& "d:/AI&SC/Clausify/backend/venv/Scripts/python.exe" -m uvicorn app.main:app --host 0.0.0.0 --port 8000`

2. Frontend (Vite on Port 5173):
`cd "d:/AI&SC/Clausify/frontend"; node "node_modules/vite/bin/vite.js"`

3. Unified Gateway (Reverse Proxy on Port 8080):
`& "d:/AI&SC/Clausify/backend/venv/Scripts/python.exe" "d:/AI&SC/Clausify/backend/gateway_8080.py"`

Running Test Suites:

Document Ingestion & Risk Scoring Suite:
`& "d:/AI&SC/Clausify/backend/venv/Scripts/python.exe" "d:/AI&SC/Clausify/backend/test_pipeline.py"`

RAG Chat, FAQ Pre-check & Legal Notice Compilation Suite:
`& "d:/AI&SC/Clausify/backend/venv/Scripts/python.exe" "d:/AI&SC/Clausify/backend/test_chat_and_notice.py"`